

Lập trình web 1- PHP&MySQL – Lab 6

Nội dung thực hành:

- Xây dựng giao diện Admin.
- Tạo Layout dùng chung.
- Xây dựng Dashboard.
- Kiểm tra giao diện.
- Phát triển Dashboard.
- Mở rộng các lớp DAO để hỗ trợ Dashboard.

Câu A. Cấu trúc đồ án (Mini Project)	2
Câu B. TẠO CƠ SỞ DỮ LIỆU (MYSQL)	3
Câu C. Xây dựng Entity (Model)	7
Câu D. Xây dựng lớp kết nối cơ sở dữ liệu (Database)	9
Câu E. Xây dựng lớp DAO (Data Access Object)	10
Câu F. Xây dựng giao diện quản trị (Admin)	16
Câu G. Phát triển thêm giao diện Dashboard	19
Câu H. Đẩy source code thay đổi lên GitHub	19

Câu A. Cấu trúc đồ án (Mini Project)

MiniShop_HoTenKhongDau/ ├── config/ ├── dao/ ├── models/ ├── services/ ├── views/ ├── admin/ └── client/ ├── uploads/ └── products/ └── categories/ └── ├── assets/ ├── index.php ├── .htaccess └── hotensv_database.sql	models: Entity (Category, Product, User...) dao: CRUD với MySQLi services: Upload ảnh, kiểm tra đăng nhập... views/admin: Giao diện quản trị/ nhân viên views/client: Giao diện người dùng assets: CSS, JS, uploads: Ảnh sản phẩm .htaccess : là file cấu hình của Apache http://localhost/MiniShop/product.php?id=10 ⇒ http://localhost/MiniShop/product/10
--	--

Câu B. TẠO CƠ SỞ DỮ LIỆU (MYSQL)

1. Tạo cơ sở dữ liệu theo tên: hotensv_database
2. Tạo các bảng : categories, brands, products, users , orders , order_details ,...

Bảng	Mô tả	Quan hệ
categories	Lưu thông tin danh mục sản phẩm (Chuột, Bàn phím, Tai nghe...).	1 - N với products. Một danh mục có nhiều sản phẩm.
brands	Lưu thông tin thương hiệu (Logitech, Dell, Asus...).	1 - N với products. Một thương hiệu có nhiều sản phẩm.
products	Lưu thông tin sản phẩm như tên, giá, số lượng, hình ảnh, danh mục, thương hiệu...	N - 1 với categories, N - 1 với brands, 1 - N với product_images, 1 - N với order_details.
product_images	Lưu nhiều hình ảnh của một sản phẩm.	N - 1 với products.
customers	Lưu thông tin khách hàng.	1 - N với orders. Một khách hàng có nhiều đơn hàng.
users	Lưu tài khoản quản trị và nhân viên.	1 - N với orders. Một nhân viên có thể tạo nhiều đơn hàng (có thể NULL).
orders	Lưu thông tin đơn hàng.	N - 1 với customers, N - 1 với users, 1 - N với order_details.
order_details	Lưu các sản phẩm thuộc từng đơn hàng.	N - 1 với orders, N - 1 với products.

3. Thiết kế cấu trúc các bảng:

Bảng categories : danh mục/ loại sản phẩm			
Tên cột	Kiểu dữ liệu	Ràng buộc	Ghi chú
id	INT	PRIMARY KEY, AUTO INCREMENT	Mã danh mục
catename	VARCHAR(100)	NOT NULL	Tên danh mục
slug	VARCHAR(150)	UNIQUE	Đường dẫn thân thiện
	<i>Tên: Chuột Logitech => Slug: chuột-logitech</i>		
image	VARCHAR(255)	NULL	Hình ảnh danh mục
description	TEXT	NULL	Mô tả danh mục
status	TINYINT	DEFAULT 1	Trạng thái (1: Hiển thị, 0: Ẩn)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Ngày tạo
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Ngày cập nhật

Lập trình web 1- PHP&MySQL – Lab 6

Bảng brands : thương hiệu			
Tên cột	Kiểu dữ liệu	Ràng buộc	Ghi chú
id	INT	PRIMARY KEY, AUTO INCREMENT	Mã thương hiệu
brandname	VARCHAR(100)	NOT NULL	Tên thương hiệu
slug	VARCHAR(150)	UNIQUE	Đường dẫn thân thiện
image	VARCHAR(255)	NULL	Logo thương hiệu
description	TEXT	NULL	Mô tả
status	TINYINT	DEFAULT 1	Trạng thái (1: Hiển thị, 0: Ẩn)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Ngày tạo
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Ngày cập nhật

Bảng products : sản phẩm			
Tên cột	Kiểu dữ liệu	Ràng buộc	Ghi chú
id	INT	PRIMARY KEY, AUTO INCREMENT	Mã sản phẩm
category_id	INT	FOREIGN KEY	Thuộc danh mục
brand_id	INT	FOREIGN KEY	Thuộc thương hiệu
praname	VARCHAR(200)	NOT NULL	Tên sản phẩm
slug	VARCHAR(150)	UNIQUE	Đường dẫn thân thiện
price	DECIMAL(10,0)	NOT NULL	Giá gốc
discount_price	DECIMAL(10,0)	NOT NULL	Giá sau giảm
quantity	INT	DEFAULT 0	Số lượng tồn kho
image	VARCHAR(255)	NULL	Hình ảnh sản phẩm
description	TEXT	NULL	Mô tả sản phẩm
status	TINYINT	DEFAULT 1	Trạng thái (1: Còn bán, 0: Ngừng bán)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Ngày tạo
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Ngày cập nhật

Lập trình web 1- PHP&MySQL – Lab 6

Bảng product_images : hình ảnh sản phẩm			
Tên cột	Kiểu dữ liệu	Ràng buộc	Ghi chú
id	INT	PRIMARY KEY, AUTO INCREMENT	Mã hình ảnh
product_id	INT	NOT NULL, FOREIGN KEY → products	Sản phẩm
image	VARCHAR(255)	NOT NULL	Đường dẫn hình ảnh
sort_order	INT	DEFAULT 0	Thứ tự hiển thị
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Ngày tạo

Bảng users : người dùng/nhân viên			
Cột	Kiểu dữ liệu	Ghi chú	Cột
id	INT	PK, AUTO_INCREMENT	id
fullname	VARCHAR(100)	Họ tên	fullname
username	VARCHAR(50)	Tên đăng nhập	username
password	VARCHAR(255)	Mật khẩu	password
email	VARCHAR(100)	Email	email
phone	VARCHAR(20)	Số điện thoại	phone
address	VARCHAR(255)	Địa chỉ	address
role	TINYINT	0: Nhân viên, 1: Quản trị	role
status	TINYINT	Trạng thái	status
created_at	DATETIME	Ngày tạo	created_at
updated_at	DATETIME	Ngày cập nhật	updated_at

Bảng customers : khách hàng			
Tên cột	Kiểu dữ liệu	Ràng buộc	Ghi chú
id	INT	PK, AUTO_INCREMENT	Mã khách hàng
fullname	VARCHAR(100)	NOT NULL	Họ và tên
phone	VARCHAR(20)	NOT NULL	Số điện thoại
email	VARCHAR(100)	NULL	Email
address	VARCHAR(255)	NULL	Địa chỉ
note	TEXT	NULL	Ghi chú

Lập trình web 1- PHP&MySQL – Lab 6

status	TINYINT	DEFAULT 1	Trạng thái
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Ngày tạo
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Ngày cập nhật

Bảng orders : đơn hàng			
Tên cột	Kiểu dữ liệu	Ràng buộc	Ghi chú
id	INT	PK, AUTO_INCREMENT	Mã đơn hàng
customer_id	INT	NOT NULL, FOREIGN KEY → customers	Khách hàng
user_id	INT	NULL, FOREIGN KEY → users	Nhân viên tạo đơn (nếu có)
order_code	VARCHAR(30)	NOT NULL, UNIQUE	Mã đơn hàng
total_amount	DECIMAL(12,2)	DEFAULT 0	Tổng tiền
note	TEXT	NULL	Ghi chú
status	TINYINT	DEFAULT 0	0: Chờ xử lý, 1: Hoàn thành, 2: Hủy
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Ngày tạo
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Ngày cập nhật
customer_id	INT	NOT NULL, FOREIGN KEY	Khách hàng

Bảng order_details : đơn hàng			
Cột	Kiểu dữ liệu	Ghi chú	Cột
id	INT	PK, AUTO_INCREMENT	id
order_id	INT	FK → orders	order_id
product_id	INT	FK → products	product_id
quantity	INT	Số lượng	quantity
price	DECIMAL(10,2)	Giá bán tại thời điểm đặt	price
subtotal	DECIMAL(12,2)	Thành tiền	subtotal
created_at	DATETIME	Ngày tạo	created_at

4. Chèn dữ liệu mẫu
 - Mỗi bảng tối thiểu 5 bản ghi
5. Xuất CSDL:
 - hotensv_database.sql
 - Lưu vào cấu trúc project

Câu C. Xây dựng Entity (Model)

1. Tạo trong thư mục **models**, tạo các lớp **Model** tương ứng với mỗi bảng trong cơ sở dữ liệu models/
 - |
 - |— Category.php
 - |— Brand.php
 - |— Product.php
 - |— ProductImage.php
 - |— Customer.php
 - |— User.php
 - |— Order.php
 - |— OrderDetail.php
2. Đặt tên lớp, thuộc tính và phương thức theo quy tắc **PascalCase** và **camelCase**:
 - **Tên lớp (Class):** sử dụng **PascalCase**, mỗi từ bắt đầu bằng chữ cái viết hoa.
Ví dụ: Category, Product, ProductImage, OrderDetail.
 - **Tên thuộc tính (Property):** sử dụng **camelCase**, từ đầu tiên viết thường, các từ tiếp theo viết hoa chữ cái đầu.
Ví dụ: categoryId, brandId, oldPrice, salePrice, createdAt, updatedAt.
 - **Tên thuộc tính (Property):** sử dụng **camelCase**, từ đầu tiên viết thường, các từ tiếp theo viết hoa chữ cái đầu.
Ví dụ: categoryId, brandId, oldPrice, salePrice, createdAt, updatedAt.
 - **Tên file:** đặt trùng với tên lớp và phân biệt chữ hoa, chữ thường.
Ví dụ: Category.php, Product.php, OrderDetail.php.
 - Không sử dụng dấu tiếng Việt, khoảng trắng hoặc ký tự đặc biệt trong tên lớp, tên thuộc tính và tên phương thức.
3. Mỗi lớp phải khai báo đầy đủ các thuộc tính tương ứng với các cột của bảng.

Bảng categories		Category.php	
Cột CSDL	Kiểu dữ liệu	Thuộc tính	Kiểu dữ liệu PHP
id	INT	\$id	int
name	VARCHAR	\$name	string
slug	VARCHAR	\$slug	string
image	VARCHAR	\$image	?string
description	TEXT	\$description	?string
status	TINYINT	\$status	int
created_at	DATETIME	\$createdAt	string
updated_at	DATETIME	\$updatedAt	string

Category.php	
<pre><?php class Category { public int \$id; public string \$name; public string \$slug; public ?string \$image; public ?string \$description; public int \$status; public string \$createdAt; public string \$updatedAt; public function __construct(string \$name = "", string \$slug = "", ?string \$image = null, ?string \$description = null, int \$status = 1) { \$this->name = \$name; \$this->slug = \$slug; \$this->image = \$image; \$this->description = \$description; \$this->status = \$status; } }</pre>	- Thuộc tính id không cần đưa vào Constructor vì được tự động tăng (AUTO_INCREMENT). - Các thuộc tính createdAt và updatedAt không cần đưa vào Constructor vì được cơ sở dữ liệu tự động tạo và cập nhật. - Trong Mini Project, các thuộc tính được khai báo ở mức public để đơn giản hóa việc truy cập dữ liệu và làm việc với lớp DAO. - Trong các dự án thực tế, các thuộc tính thường được khai báo ở mức private và truy cập thông qua Getter/Setter để đảm bảo tính đóng gói (Encapsulation).

<pre>} }</pre>	
--------------------	--

4. Thực hiện tương tự cho các bảng còn lại, mỗi bảng tương ứng với một lớp Model, khai báo đầy đủ các thuộc tính và Constructor

- └── Brand.php
- └── Product.php
- └── ProductImage.php
- └── Customer.php
- └── User.php
- └── Order.php
- └── OrderDetail.php

Câu D. Xây dựng lớp kết nối cơ sở dữ liệu (Database)

1. Tạo lớp Database

config/ └── Database.php	Lớp Database có nhiệm vụ quản lý việc kết nối đến cơ sở dữ liệu MySQL bằng MySQLi .
-----------------------------	--

2. Khai báo thuộc tính

Thuộc tính	Kiểu dữ liệu	Giá trị	Ghi chú
\$host	string	localhost	Địa chỉ máy chủ CSDL
\$database	string	hotensv_database	Tên cơ sở dữ liệu
\$username	string	root	Tài khoản đăng nhập
\$password	string	""	Mật khẩu
\$conn	mysqli	NULL	Đối tượng kết nối MySQL

<pre><?php class Database { protected string \$host = "localhost"; protected string \$database = "hotensv_database"; protected string \$username = "root"; protected string \$password = "";</pre>	try...catch là cơ chế dùng để bắt và xử lý ngoại lệ (Exception) trong chương trình.
--	--

<pre>protected mysqli \$conn; public function __construct() { \$this->connect(); } // Kết nối cơ sở dữ liệu protected function connect(): void { try { \$this->conn = new mysqli(\$this->host,\$this->username,\$this->password, \$this- >database); if (\$this->conn->connect_errno) { throw new Exception("Kết nối DB thất bại: " . \$this->conn- >connect_error); } \$this->conn->set_charset("utf8mb4"); } catch (Exception \$e) { throw new Exception(\$e->getMessage()); } } // Trả về đối tượng kết nối MySQLi public function getConnection(): mysqli { return \$this->conn; } /* Đóng kết nối public function close(): void { if (isset(\$this->conn)) { \$this->conn->close(); } } }</pre>	Hạn chế lạm dụng try...catch; Đối với các thao tác với cơ sở dữ liệu (kết nối, truy vấn, thêm, sửa, xóa), nên sử dụng try...catch để xử lý lỗi và tránh làm chương trình bị dừng đột ngột.
---	---

Câu E. Xây dựng lớp DAO (Data Access Object)

1. Tạo thư mục, Tạo các lớp DAO
dao/

— BaseDAO.php	// chứa phương thức dùng chung các lớp DAO
— CategoryDAO.php	// categories
— BrandDAO.php	// brands
— ProductDAO.php	// products, product_images
— CustomerDAO.php	// customers
— UserDAO.php	// users
— OrderDAO.php	// orders, order_details

2. BaseDAO.php

- Là lớp cơ sở cho các lớp DAO.
- Chứa các phương thức dùng chung (CRUD, thực thi câu lệnh SQL, xử lý Prepared Statement...) để các lớp DAO kế thừa và tái sử dụng.
- Kế thừa từ lớp Database

BaseDAO.php	
<pre> <?php require_once __DIR__ . "../config/Database.php"; class BaseDAO extends Database { public function __construct() { parent::__construct(); // Thực thi câu lệnh SELECT protected function executeQuery(string \$sql): mysqli_result false { return \$this->conn->query(\$sql); } // Chuẩn bị câu lệnh Prepared Statement protected function prepare(string \$sql): mysqli_stmt false { return \$this->conn->prepare(\$sql); } // Bắt đầu Transaction protected function beginTransaction(): void { \$this->conn->begin_transaction(); } } </pre>	parent::__construct(); : Gọi Constructor của lớp cha để khởi tạo các thuộc tính và phương thức được kế thừa. protected: Chỉ truy cập trong lớp và các lớp kế thừa. Xem lại tài liệu: public, private, protected

```
// Xác nhận Transaction
protected function commit(): void
{
    $this->conn->commit();
}

// Hủy Transaction
protected function rollback(): void
{
    $this->conn->rollback();
}

// Đóng kết nối
public function close(): void
{
    $this->conn->close();
}
}
```

3. CategoryDAO.php

CategoryDAO

```
<?php
require_once __DIR__ . "/BaseDAO.php";
require_once __DIR__ . "../models/Category.php";

class CategoryDAO extends BaseDAO
{
    public function __construct()
    {
        parent::__construct();
    }

    // Lấy tất cả danh mục
    public function getAll(): array
    {
        $list = [];
        try {
            // Hạn chế dùng SELECT * (nên chỉ định rõ cột cần truy vấn)
            $sql = "SELECT * FROM categories ORDER BY catename";
            $result = $this->executeQuery($sql);
            while ($row = $result->fetch_assoc()) {
                $category = new Category(
                    $row["catename"],
```

```
        $row["slug"],
        $row["image"],
        $row["description"],
        $row["status"]
    );
    $category->id = $row["id"];
    $category->createdAt = $row["created_at"];
    $category->updatedAt = $row["updated_at"];
    $list[] = $category;
    }
} catch (Exception $e) {
    throw $e;
}
return $list;
}
```

// Tìm theo ID

```
public function findById(int $id): ?Category
{
    try {

        $sql = "SELECT * FROM categories WHERE id=?";
        $stmt = $this->prepare($sql);
        $stmt->bind_param("i", $id);
        $stmt->execute();
        $result = $stmt->get_result();
        if ($row = $result->fetch_assoc()) {
            $category = new Category(
                $row["name"],
                $row["slug"],
                $row["image"],
                $row["description"],
                $row["status"]
            );
            $category->id = $row["id"];
            $category->createdAt = $row["created_at"];
            $category->updatedAt = $row["updated_at"];
            return $category;
        }
    } catch (Exception $e) {
        throw $e;
    }
    return null;
}
```

// Thêm danh mục

```
public function insert(Category $category): bool
{
    try {

        $sql = "INSERT INTO categories(catename,slug,image,description,status)
                VALUES(?,?,?,?,?)";
        $stmt = $this->prepare($sql);
        $stmt->bind_param(
            "ssssi",
            $category->name,
            $category->slug,
            $category->image,
            $category->description,
            $category->status
        );
        return $stmt->execute();
    } catch (Exception $e) {
        throw $e;
    }
}
```

// Cập nhật danh mục

```
public function update(Category $category): bool
{
    try {

        $sql = "UPDATE categories
                SET
                    catename=?,
                    slug=?,
                    image=?,
                    description=?,
                    status=?
                WHERE id=?";

        $stmt = $this->prepare($sql);
        $stmt->bind_param(
            "ssssi",
            $category->name,
            $category->slug,
            $category->image,
            $category->description,
```

```
        $category->status,  
        $category->id  
    );  
    return $stmt->execute();  
} catch (Exception $e) {  
    throw $e;  
}  
}  
}  
  
// Xóa danh mục  
public function delete(int $id): bool  
{  
    try {  
        $sql = "DELETE FROM categories WHERE id=?";  
        $stmt = $this->prepare($sql);  
        $stmt->bind_param("i", $id);  
        return $stmt->execute();  
    } catch (Exception $e) {  
        throw $e;  
    }  
}  
}
```

4. Dựa trên lớp CategoryDAO đã xây dựng, thực hiện tương tự cho các lớp DAO còn lại. Mỗi lớp DAO:

- Kế thừa BaseDAO.
- Xây dựng Constructor.
- Thực hiện đầy đủ các phương thức CRUD ((getAll(), findById(), insert(), update(), delete()))
- Sử dụng Prepared Statement cho các câu lệnh có tham số.
- Ánh xạ dữ liệu thành đối tượng Model tương ứng.
- Sử dụng try...catch để xử lý ngoại lệ

dao/

— BaseDAO.php	// chứa phương thức dùng chung các lớp DAO
— CategoryDAO.php	// categories
— BrandDAO.php	// brands
— ProductDAO.php	// products, product_images
— CustomerDAO.php	// customers

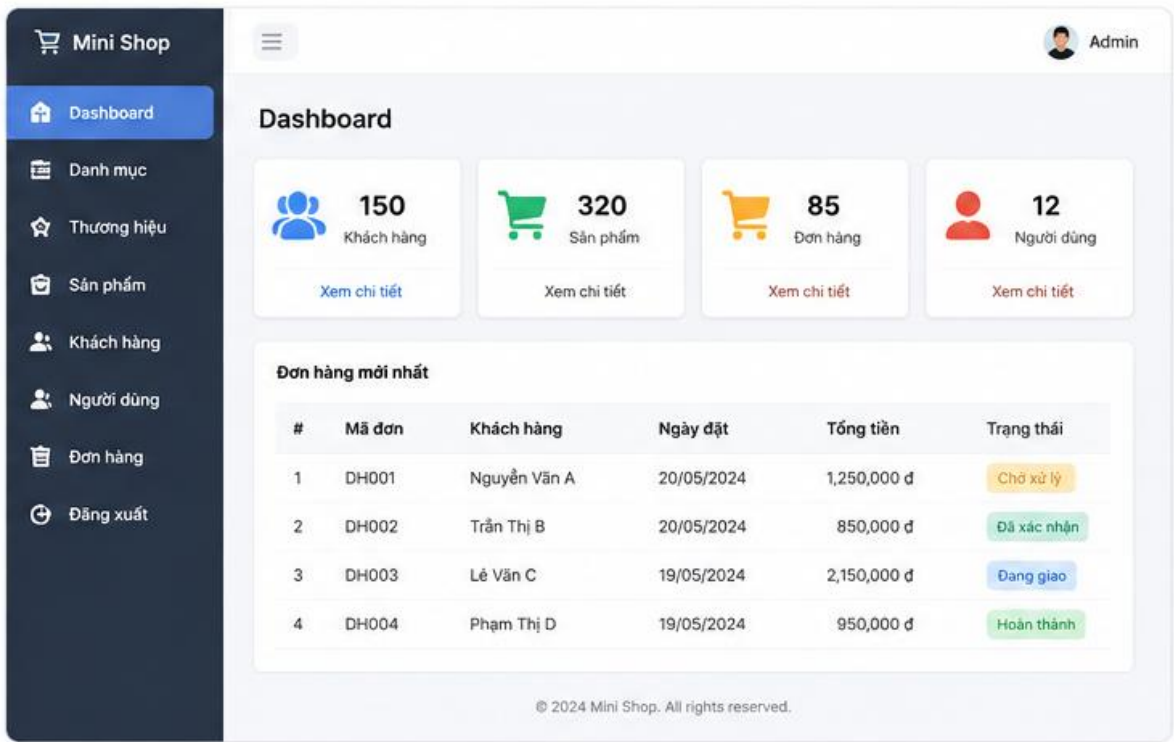
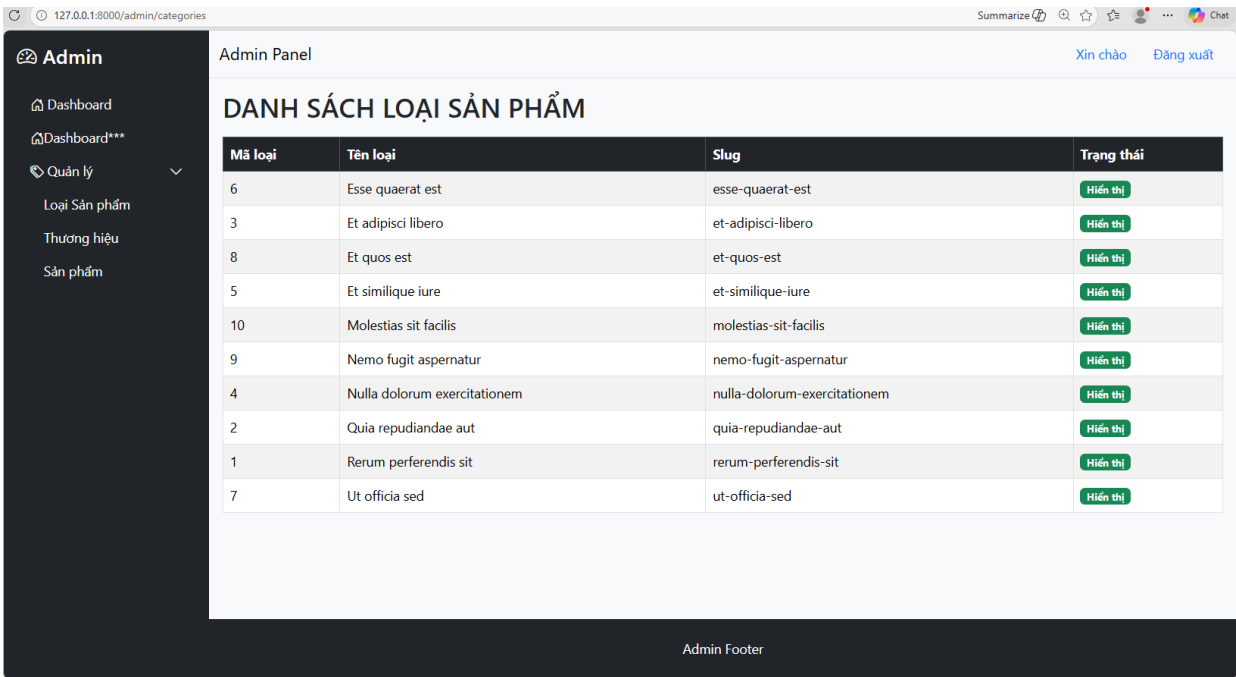
UserDAO.php

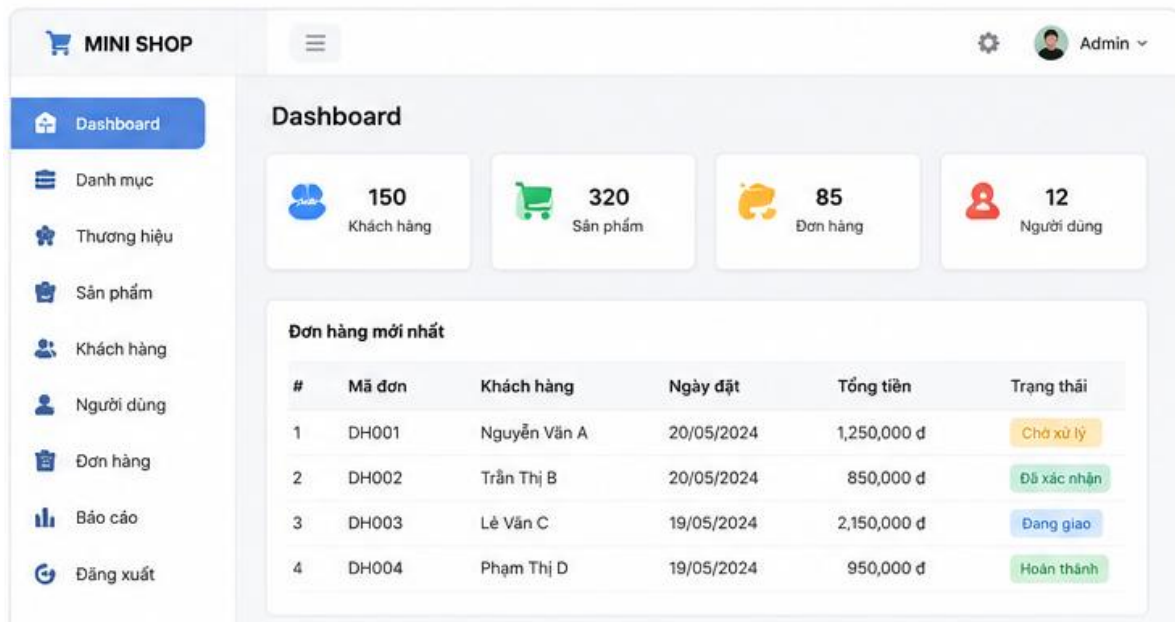
OrderDAO.php

// users

// orders, order_details

Câu F. Xây dựng giao diện quản trị (Admin)





1. Tạo thư mục giao diện quản trị

views/

└─ admin/

└─ layouts/

└─ header.php // Hiển thị logo, tiêu đề và thông tin người dùng.

└─ sidebar.php // Hiển thị menu quản trị.

└─ footer.php // Hiển thị thông tin cuối trang.

└─ master.php // Giao diện chính, ghép Header + Sidebar + Content + Footer.

└─ dashboard.php // Trang tổng quan của hệ thống quản trị.

└─ categories/

└─ index.php // Danh sách danh mục.

└─ brands/

└─ index.php // Danh sách thương hiệu.

└─ products/

└─ index.php // Danh sách sản phẩm.

└─ customers/

└─ index.php // Danh sách khách hàng.

└─ users/

└─ index.php // Danh sách tài khoản người dùng.

└─ orders/

└─ index.php // Danh sách đơn hàng.

master.php	
<pre><?php include "header.php"; ?> <div class="container-fluid"> <div class="row"> <?php include "sidebar.php"; ?> <div class="col"> <?= \$content ?> </div> </div> </div> <?php include "footer.php"; ?></pre>	<pre><?= \$content ?></pre> Hiển thị nội dung của từng trang (Dashboard, Category, Product, ...).
▪ Phần Header và Sidebar: Sinh viên có thể tham khảo giao diện phù hợp trên Internet hoặc tự thiết kế theo ý tưởng của mình (Có thể tham khảo bootstrap kết hợp CSS thuần) ▪ Riêng phần Header, bắt buộc sử dụng biến \$pageTitle để hiển thị tiêu đề của trang trong thẻ <title>. <title><?= \$pageTitle ?></title>	
Lưu ý: Chỉ nên tham khảo và sao chép từng thành phần (Header, Sidebar, Card,...) để tự xây dựng giao diện, không sao chép toàn bộ Template. Ưu tiên sử dụng các thành phần Bootstrap cơ bản.	

dashboard.php	
<pre>\$pageTitle = "Dashboard"; ob_start(); ?> <h2>Dashboard</h2> <div class="alert alert-success"> Chào mừng bạn đến với hệ thống quản trị Mini Shop. </div> <?php \$content = ob_get_clean(); include "layouts/master.php";</pre>	<pre>ob_start();</pre> Bắt đầu Output Buffering (bộ nhớ đệm đầu ra). Mọi nội dung HTML phía sau sẽ chưa được xuất ra trình duyệt mà được lưu tạm vào bộ nhớ. <pre>ob_get_clean();</pre> Lấy toàn bộ nội dung đã lưu trong Output Buffer và gán vào biến \$content, sau đó xóa bộ nhớ đệm.

- Chạy và kiểm tra chương trình

Câu G. Phát triển thêm giao diện Dashboard

- Hiển thị tổng số Danh mục, Thương hiệu, Sản phẩm, Khách hàng và Đơn hàng.
- Hiển thị 05 sản phẩm mới nhất.
- Hiển thị 05 đơn hàng mới nhất.
- Tùy chỉnh và phát triển giao diện Dashboard phù hợp với hệ thống.
- Xây dựng thêm các phương thức trong các lớp DAO để hỗ trợ truy vấn và hiển thị dữ liệu trên Dashboard (không nên tạo DashboardDAO.php – gọi từ các lớp DAO đã dựng)

Câu H. Đẩy source code thay đổi lên GitHub

- Mở terminal tại thư mục chứa project
- Thêm source vào Git, dùng lệnh : git add .
- Commit source: git commit -m "add MiniProject - 1"
- Đẩy source lên GitHub: git push origin main
- Kiểm tra lại trên GitHub: source code đã được cập nhập